Joint Prompt and KV-Cache Compression for Multi-Turn LLM Agents

Wenqiang Luo
City University of Hong Kong
Hong Kong, China
wenqialuo4-c@my.cityu.edu.hk

Fangkai Yang
Microsoft
Beijing, China
fangkaiyang@microsoft.com

Yu Kang
Microsoft
Shanghai, China
kay@microsoft.com

Lu Wang
Microsoft
Beijing, China
wlu@microsoft.com

Pu Zhao
Microsoft
Beijing, China
Pu.Zhao@microsoft.com

Mingzheng Li
Microsoft
Beijing, China
mingzhengli@microsoft.com

Jimeng Sun
Microsoft
Beijing, China
jimengsun@microsoft.com

Jianfeng Liu
Microsoft
Beijing, China
jianfengliu@microsoft.com

Jacky Keung
City University of Hong Kong
Hong Kong, China
Jacky.Keung@cityu.edu.hk

Qingwei Lin
Microsoft
Beijing, China
qlin@microsoft.com

Saravan Rajmohan
Microsoft
Redmond, WA, USA
saravan.rajmohan@microsoft.com

Abstract

Across multi-turn agent interactions with Web sites, services, and software tools, accumulated history increases prompt length and KV-cache memory. Independent compression overlooks both the coupling between these views and its effects on later turns. We formulate joint compression as a sequential decision problem over persistent memory units. LACING uses stable anchors to link each unit's text to its KV entries across heads. Conditioned on textual features and evolving KV state, one categorical policy selects preservation, text deletion, or headwise KV reduction. We train the policy online from task-level rollout groups to minimize cumulative backbone GPU cost subject to utility retention relative to a fixed full-cache reference. Separate cost and utility credit mechanisms assign downstream savings and progress regressions to individual compression actions. We evaluate LACING with two backbones on Terminal-Bench 2.1 and a 200-task subset of SWE-bench Verified. Across all four settings, it achieves higher task success than the four compression baselines. Relative to full-cache, it reduces backbone GPU-seconds by 7.5–14.0% and mean peak GPU-resident KV memory by 26–55%, while preserving 98.9% of full-cache's overall task-success rate. In additional shared-context tests on SCBench, LACING retains over 92% of full-cache successes on compression-sensitive key-value and code retrieval tasks.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; **Intelligent agents**.

Keywords

Large Language Models, LLM Agents, Prompt Compression, KV-Cache Compression, Reinforcement Learning

ACM Reference Format:
Wenqiang Luo, Fangkai Yang, Yu Kang, Lu Wang, Pu Zhao, Mingzheng Li, Jimeng Sun, Jianfeng Liu, Jacky Keung, Qingwei Lin, and Saravan Rajmohan. 2027. Joint Prompt and KV-Cache Compression for Multi-Turn LLM Agents. In *Proceedings of The ACM Web Conference 2027 (WWW '27)*. ACM, New York, NY, USA, 16 pages.

1 Introduction

LLM agents solve tasks through sequences of reasoning, actions, and observations from Web sites, services, and software tools [38]. Each interaction adds information that later decisions may need. For example, a constraint encountered on one page can remain relevant when the agent acts on another page several steps later. Maintaining this history also grows the prompt and its key–value (KV) cache. Even with cache reuse, newly appended inputs attend to an expanding context, and subsequent generations access more cached states [4]. The resulting compute and memory costs accumulate over the task. Efficient serving of these agents requires retaining useful history without carrying its full inference cost through every later interaction.

Text and KV compression act on the same history, but each decides what to remove using its own signals. Text compression prunes tokens or interaction records [24, 34, 40], with token selection guided by perplexity [8] or learned retention scores [24]. KV compression uses attention patterns across heads or learned policies [2, 4, 17, 23] to select cached states. These methods separately decide which text to remove and which KV entries to remove. They do not compare whether the same content should be kept in full,

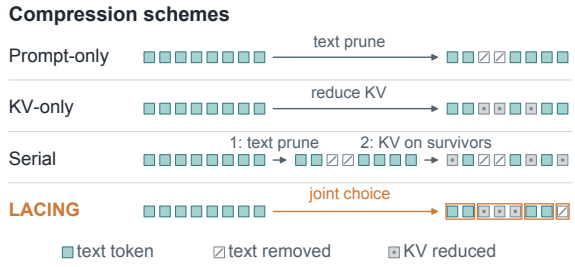


Figure 1: Compression routes over the same agent history. Orange outlines mark LACING's memory units.

deleted from the text, or retained with fewer KV entries. Simply applying the two compressors in sequence leaves this choice split between their separate signals and policies. What is missing is a shared representation and a single decision space in which all three options are compared according to their value for the task and their inference cost.

Previous work has combined text compression with KV compression to obtain further compression gains [20]. In a serial pipeline, the order of decisions creates a further restriction. The KV compressor can act only on content retained by the text compressor. Consider a fact marked for text deletion that may still be needed in a later interaction. Retaining its text while removing part of its headwise KV support could preserve useful information and still save inference cost. Once the text stage deletes that fact, the KV stage cannot substitute this partial reduction for the earlier deletion. We therefore introduce LACING, a joint prompt and KV-cache compression policy for multi-turn agents. For each memory unit, a shared policy uses the same text–KV representation and pre-compression state to choose among `keep`, `drop`, and `evict` before committing an action. These alternatives compete within one decision space, with task-level feedback guiding their cumulative cost and utility trade-offs. Figure 1 contrasts single-view, serial, and joint compression on the same agent history.

Prompt compression often controls retained token counts or compression ratios [8, 24], whereas KV compression controls cache size [17]. Text deletion and KV reduction change the computation and memory traffic required by model inference. These changes affect the GPU workload during prefill, decoding, and any required recomputation. We express this backbone inference cost in GPU-seconds, providing a common unit for comparing the two forms of compression. For agents acting across pages and services, a compression decision can save inference work across later interactions while also changing subsequent outputs and tool calls. Losing information needed later may lead to repeated work or reduce task utility. Therefore, we formulate joint compression as minimizing cumulative trajectory GPU cost subject to preserving task utility relative to a fixed full-cache reference.

LACING organizes history into persistent memory units, each with a stable anchor linking its text to the corresponding KV entries. Anchors maintain this correspondence as text positions shift and cache coverage changes. A shared policy conditions on each unit's textual features and evolving KV state to choose one action per unit. Its actions are `keep`, `drop` for text deletion, and `evict` for KV reduction. These alternatives compete within a single decision space. All units are scored from the same pre-compression state before the selected actions update the history and cache.

We train this policy from groups of online task rollouts while freezing the backbone. Task-level cost and utility feedback reflects the combined effects of many compression decisions, giving limited guidance on their individual contributions. We use separate cost and utility credit mechanisms to provide action-level learning signals. The cost channel accumulates an action's marginal savings over the observed rollout. To compute these savings, we restore its removed text length or KV bytes in the cost calculation. Generated tokens and other actions are held fixed. The utility channel uses persistent regressions relative to a full-cache progress curve to assign penalties to earlier compression actions.

We examine this shared runtime problem in interactive software-agent workflows, which generate histories of tool actions and observations and provide verifiable task outcomes. We evaluate LACING on 89 Terminal-Bench 2.1 tasks [21] and a 200-task subset of SWE-bench Verified [10]. We use Qwen3-Coder-30B-A3B-Instruct and Devstral-Small-2-24B as backbones. Comparisons include full-cache, LLMLingua-2, SnapKV, their serial composition, and AgentDiet.

Across all four model–benchmark settings, LACING improves task success by 0.5–10.0 percentage points over the best of the four compression baselines. Relative to full-cache, it reduces reported backbone GPU-seconds by 7.5–14.0% and mean peak GPU-resident KV memory by 26–55%. Full-cache success rates are preserved or improved on Qwen and retained to within 1.2 percentage points on Devstral. Action, feature, and credit ablations examine the roles of joint selection, cross-view information, and trajectory-level learning.

The learned text–KV allocation varies across backbones. On Qwen, localized credit lowers cost and improves utility preservation over uniform credit. Shared-context tests on SCBench further show the strongest utility preservation among tested compressors when retrieving compression-sensitive JSON records, strings, and code.

Our contributions are as follows.

- We introduce LACING, a joint prompt and KV-cache compression policy whose persistent anchors support unified action selection under a trajectory-level cost objective and utility-preservation constraint.
- We develop separate cost and utility credit mechanisms that connect unit-level decisions to cumulative savings and task-progress regressions.
- We demonstrate cost and memory reductions at near-full-cache success across two agent backbones and two benchmarks, complemented by ablations, trajectory analyses, and shared-context retrieval tests.

2 Related Work

Prompt Compression. Prompt compression controls the representation supplied to the decoder. LLMLingua uses perplexity-guided token selection, while LongLLMLingua adds question-conditioned relevance [8, 9]. LLMLingua-2 learns a retention classifier from distilled compression data [24]. AutoCompressors and ICAE instead learn compact continuous representations [1, 5]. Extractive

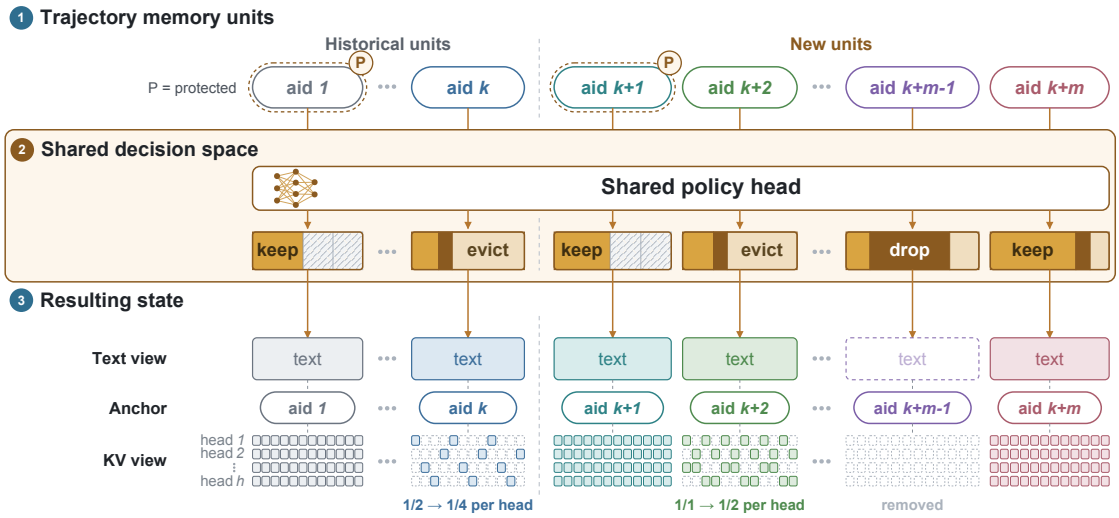


Figure 2: Overview of joint prompt and KV-cache compression. "New units" have completed their one-decision-step initial protection. Hatching marks masked actions. Dashed text boxes and KV cells mark removal.

methods preserve a readable token sequence, whereas continuous compression requires a learned interface to the decoder. In both cases, subsequent cache use follows the compressed input.

KV Cache Compression and Management. KV compression uses attention statistics, reconstruction, or learned retention policies. FastGen profiles head-specific attention, while SnapKV selects positions using an observation window [4, 17]. KVzip scores reusable cache entries through context reconstruction [13]. KVP learns token rankings from future-attention rewards, while DuoAttention and RLKV learn head configurations using hidden-state distillation and generated-answer correctness, respectively [2, 23, 33]. MemoRAG trains compressed memory for retrieval using generation-quality feedback [25]. Complementary systems separate token propagation from KV retention in FastKV [11], combine text and KV compression for transmission in CacheGen [20], and jointly optimize compression and storage-tier placement in EvicPress [3]. These mechanisms control retention, propagation, or storage rather than selecting between text deletion and KV reduction per unit.

Context Management for Web and Tool Agents. WebArena studies multi-step Web tasks in which later actions depend on earlier observations [44]. AgentOccam condenses webpage observations and selectively replays interaction history [37]. Agent Workflow Memory induces reusable workflows from past trajectories to guide subsequent tasks [30]. These approaches organize the information available to the agent across interactions.

Agent memory management couples compression to ongoing task execution. AgentDiet removes redundant and outdated history [34]. ACON refines compression guidelines using task-outcome contrasts [12]. ReSum periodically summarizes Web-search histories. ReSum-GRPO broadcasts trajectory-level advantages across the resulting segments [32]. MemAct jointly learns task behavior and edits to addressable memory [40]. Its dynamic-context optimization assigns each reconstructed segment the trajectory's group-relative advantage. Our learning rule instead separates action-level credit for accumulated GPU-cost savings from credit for utility regressions relative to full-cache progress.

Persistent KV management exploits interaction structure. EpiCache maintains episode-specific caches, CommitKV evaluates event pages across commit transitions, and AgentKV uses reasoning, action, and tool phases to guide retention [7, 14, 19]. These methods organize cache decisions around conversational or agent state. LACING links each unit's current text and KV support through a persistent anchor, making preservation, text deletion, and KV reduction alternatives under one policy. The policy targets lower trajectory-level GPU cost under a utility-preservation constraint.

3 Problem Formulation

We study joint compression of text and KV memory for multi-turn LLM agents. Each decision changes the resources available to subsequent generations. The objective is to reduce cumulative backbone inference cost subject to a task-utility constraint.

3.1 Agent State and Cross-View Memory

Let t index compression decisions after completed agent responses. The pre-decision state consists of the append-only canonical history H_t, the retained text P_t, and the active per-layer KV state $\mathcal{K}_t = \{(K_t^\ell, V_t^\ell)\}_{\ell=1}^{L}$. The history preserves each token's original content and coordinates. Inference uses P_t and $\mathcal{K}_t$.

A superscript + denotes the state committed by compression, with retained views P_t^+ and $\mathcal{K}_t^+$ and unchanged history H_t. The next interaction appends its input and generated response to this state, yielding H_{t+1}, P_{t+1}, and $\mathcal{K}_{t+1}$.

We partition H_t into non-overlapping canonical intervals $I_i = [b_i, e_i)$ and assign each interval a permanent identifier aid_i. The

pair $\alpha_i = (\mathrm{aid}_i, I_i)$ is a permanent cross-view memory anchor. The identifier tracks the unit, and I_i locates its original tokens. At decision t, $J_{i,t}$ is the unit's current interval in P_t. For KV head h, $Z^h_{i,t} \subseteq I_i$ contains the canonical positions whose KV entries remain active. A head uses the same support across transformer layers, while different heads may retain different positions. The unit state is

$$m_{i,t} = (\alpha_i, J_{i,t}, \mathbf{Z}_{i,t}), \qquad \mathbf{Z}_{i,t} = (Z^1_{i,t}, \ldots, Z^{H_{\mathrm{KV}}}_{i,t}). \tag{1}$$

An active unit satisfies $\bigcup_h Z^h_{i,t} = I_i$. Thus, every original token has KV support in at least one head. Text compaction can change $J_{i,t}$, while the anchor remains fixed. This state separates persistent identity, current text location, and cache membership.

3.2 Joint Actions and Trajectory Objective

For every active unit, the policy selects one action from $\mathcal{A} = \{\texttt{keep}, \texttt{drop}, \texttt{evict}\}$. `Keep` preserves the current unit state. `Drop` retires the unit: $J^+_{i,t} = \perp$ (no retained text interval) and every $Z^{h,+}_{i,t} = \emptyset$. `Evict` preserves the text and reduces headwise KV support subject to cross-head token coverage. Protected units admit only `keep`. The two compression options compete with `keep` in one categorical policy, giving joint control over text and KV resources.

For task $\xi \sim \mathcal{D}$, let $\tau \sim \pi_\theta(\cdot \mid \xi)$ be the resulting agent trajectory, with cost $C(\tau)$ and utility $\mathcal{U}(\tau)$. Cost is cumulative backbone inference in GPU-seconds, including prefill, incremental append, decode, and any incurred recomputation. Utility measures task completion quality. For each task, a fixed full-cache trajectory provides the reference utility $\mathcal{U}_{\mathrm{full}}(\xi)$. With an allowed utility loss $\epsilon \geq 0$, we optimize

$$\min_\theta \ \mathbb{E}_{\xi,\tau}[C(\tau)] \quad \text{s.t.} \quad \mathbb{E}_{\xi,\tau}[\mathcal{U}_{\mathrm{full}}(\xi) - \mathcal{U}(\tau)] \leq \epsilon. \tag{2}$$

Both quantities cover the complete trajectory because a local action can change future generation, context growth, and cache use.

4 Methodology

4.1 Method Overview

Figure 2 illustrates one compression step of Lacing. After each response, newly appended history is segmented into anchored memory units. Each unit is represented by its creation-time content and its subsequent use in the text and KV views. A shared policy selects `keep`, `drop`, or `evict` from the feasible actions. The executor applies these decisions together, and the next interaction continues from the resulting compressed state.

The learning signal comes from what happens after compression. We collect task trajectories with the current policy and compare them with a fixed full-cache reference. Cost attribution measures the future savings associated with each action, while progress regressions localize utility penalties. These separate credit signals train the shared policy head with the backbone frozen.

4.2 Anchored Memory and Joint Representation

We separate each appended interaction into task and control content, reasoning, tool actions, and observations, then map these fields to token intervals. Tool-call syntax is separated from command payloads, and each command remains atomic even when it exceeds the target unit length. Reasoning and observations target 32 tokens, with an eight-token boundary-search tolerance. Fragments below eight tokens are merged when structural boundaries permit.

System and task instructions, harness controls, protocol syntax, and submission markers are permanently protected. Open command–response pairs are also protected. New units must `keep` at their creation-time decision and become eligible at the next decision unless another protection rule applies. Appendix A.1 gives segmentation thresholds and lifecycle rules.

Each unit has four feature components. *(1) Creation-time representation in 512 dimensions.* Hidden, key, value, and position statistics are pooled when the unit is appended. A frozen projection produces a vector computed once and stored under aid_i throughout its lifetime. *(2) Prompt-state features in 5 dimensions.* These track later-unit overlap and subsequent writes to the same paths. *(3) KV-demand features in 9 dimensions.* These summarize later observed queries' attention to surviving KV cells by KV head as a demand proxy [17, 41], alongside query counts and window length. Two availability indicators distinguish missing query observations from observed low attention. *(4) Lifecycle features in 3 dimensions.* These track unit age, surviving KV ratio, and prior evictions. The last three components form the 17-dimensional current state vector. We concatenate it with the fixed creation-time representation.

$$\phi_{i,t} = [\phi^{\mathrm{create}}_i; \phi^{\mathrm{state}}_{i,t}] \in \mathbb{R}^{529}. \tag{3}$$

All features use history available at decision t. The backbone, feature calibration, and projection remain frozen. Appendix A.2 gives the pooling operations and complete schema.

4.3 Joint Policy and Stateful Execution

Shared policy. A shared MLP h_θ maps each unit representation to one logit per action. Let $\mathcal{A}_{i,t}$ be the feasible actions for unit i. Protected units admit only `keep`. Eviction additionally requires at least eight surviving positions per head, attention evidence from later queries, and a feasible cross-head coverage assignment. We apply softmax over the feasible logits, assigning zero probability to infeasible actions.

$$\pi_\theta(a \mid \phi_{i,t}, \mathcal{A}_{i,t}) = \frac{\mathbf{1}[a \in \mathcal{A}_{i,t}] \exp h_\theta(\phi_{i,t})_a}{\sum_{a' \in \mathcal{A}_{i,t}} \exp h_\theta(\phi_{i,t})_{a'}}. \tag{4}$$

All unit features and feasibility masks are computed from the same pre-compression state. Selected actions are committed together.

Stateful execution. (1) Evict. The executor halves each head's KV support while retaining every token in at least one head. A coverage-constrained attention ranking selects the retained positions, with each head's selection shared across layers for a common layout.

(2) Drop. The executor deletes the unit's text and remaining KV entries, then compacts the retained text.

Unlike H_2O's streaming implementation [41], which reapplies RoPE at compact cache positions, we physically pack surviving KV entries while preserving their canonical positions and existing RoPE encodings. Removed positions remain logical gaps, and new suffix tokens follow the canonical history. Generation continues from the surviving cache without a full re-prefill.

Consider an eligible 32-token unit with full support in eight KV heads. An eviction retains 16 positions per head, reducing its head–token cells from 256 to 128 per layer while their union still covers all 32 tokens. If an earlier unit is dropped in the same step, this

unit's current text span shifts left. Its aid, canonical interval, and surviving KV position labels remain unchanged. Appendix A.3 gives the policy architecture, eligibility masks, and headwise selection and packing rules.

4.4 Trajectory Feedback and Credit Assignment

Resource savings accumulate over later generations, while the loss of useful information can affect task progress several agent steps later. We assign cost credit over subsequent forward events and utility credit through progress comparisons at agent steps.

Trajectory feedback. For each training task, we reuse one fixed full-cache trajectory τ^{ref}. It provides cost C^{ref}, utility U^{ref}, and a progress curve. We sample K trajectories τ_j from the current policy. Each trajectory's backbone cost is $C_j = \sum_{e \in \mathcal{E}_j} c(F_{j,e})$, where $\mathcal{E}_j$ is its set of forward events and $F_{j,e}$ records the resources used by event e. The stage-wise roofline model [31] maps these resources to GPU-seconds. Calibration details are provided in Appendix B.3.

At scheduled agent steps t, a frozen LLM judge assigns each execution prefix a progress score $p_j(t) \in [0, 1]$. The score measures progress toward the task's completion requirements. Candidate and reference prefixes are scored independently, and curves are compared at the same agent steps with terminal scores held fixed after termination.

Terminal training utility combines official success with terminal progress. For a trajectory ending at step T_j, we use

$$\begin{aligned} U_j &= \lambda_{\text{succ}} \mathbf{1}[\tau_j \text{ succeeds}] + (1 - \lambda_{\text{succ}}) p_j(T_j), \\ g_j &= [U^{\text{ref}} - \epsilon - U_j]_+. \end{aligned} \tag{5}$$

Here λ_{succ} weights official success, $[x]_+ = \max(x, 0)$, and the reference uses the same utility definition. The utility-constraint violation g_j measures the shortfall below $U^{\text{ref}} - \epsilon$. It is zero when this target is met. This mixture instantiates $\mathcal{U}$ in Equation 2 for training. Evaluation utility is official verifier success. Appendix B.1 gives the judge protocol and parameter values.

Cost credit. For decision $r = (i, t)$, let $\mathcal{E}_{j,r}$ contain the forward events after the action and before the next non-keep action on the same unit, or through termination. The cost saving is the subsequent backbone GPU cost with `keep` at r, minus the cost with the recorded action. Both costs are calculated from saved resource traces, holding generated tokens and all other actions fixed. To construct the `keep` case, we add back only the KV volume removed at r and, for `drop`, the removed text length. Let $F_{j,e}^{\text{keep}(r)}$ denote the resulting resource record for event e. The cumulative saving is

$$s_{j,r} = \sum_{e \in \mathcal{E}_{j,r}} \left[c(F_{j,e}^{\text{keep}(r)}) - c(F_{j,e}) \right] \geq 0. \tag{6}$$

The calculation follows the recorded trajectory. Its horizon assigns each interval between successive compressions of a unit to the decision that created that state, crediting savings across subsequent generations.

Let $\mathcal{D}_j$ be the set of trainable unit-level decision events $r = (i, t)$ in trajectory j, excluding protected decisions, and let $N_j = |\mathcal{D}_j|$. The same unit can contribute decisions at multiple steps. For positive total marginal savings, we normalize the cost weights to have mean one.

$$w_{j,r} = \frac{N_j s_{j,r}}{\sum_{r' \in \mathcal{D}_j} s_{j,r'}}. \tag{7}$$

These weights emphasize actions with larger cumulative savings.

Utility credit. We compare candidate and reference progress using $d_j(t) = p_j(t) - p^{\text{ref}}(t)$. Requiring a persistent regression filters isolated score fluctuations. Its first point defines t_j^*. The window $\mathcal{W}_j$ localizes credit to eligible trainable `drop` and `evict` decisions in the preceding W agent steps. For a nonempty window, we allocate utility credit as

$$v_{j,r} = \frac{\gamma^{t_j^* - t_r - 1}}{\sum_{r' \in \mathcal{W}_j} \gamma^{t_j^* - t_{r'} - 1}}, \quad r \in \mathcal{W}_j, \tag{8}$$

where t_r is the action's decision step and $0 < \gamma \leq 1$. For this localized allocation, weights sum to one over the window and are zero outside it. A decay $\gamma < 1$ gives more weight to recent compression actions. `Keep` receives no utility penalty. Appendix A.4 specifies regression detection, eligibility, fallback allocations, and the zero-marginal cost-credit convention.

4.5 Utility-Constrained Online Learning

We use a credit-weighted policy-gradient surrogate to learn from these signals. Define $R_j = -C_j / C^{\text{ref}}$ for each current-policy rollout. We subtract the group mean from the K rollout rewards and divide by their group standard deviation to obtain A_j, following GRPO's group-relative reward normalization [28]. The reference cost scales R_j. Group statistics use only the policy rollouts. Lower-cost trajectories have larger advantages.

For a decision $r = (i, t)$ in trajectory j, write its recorded representation and feasible action set as $\phi_{j,r}$ and $\mathcal{A}_{j,r}$. The cost channel weights this trajectory advantage by each action's cumulative savings. The utility channel subtracts a penalty proportional to the trajectory's violation and its localized credit. With dual variable $\mu_\nu \geq 0$ at training update ν, we minimize

$$\begin{aligned} \mathcal{L}_{\text{sur}}(\theta; \mu_\nu) = -\frac{1}{K} \sum_{j=1}^{K} \frac{1}{N_j} \sum_{r \in \mathcal{D}_j} \\ \left(A_j w_{j,r} - \mu_\nu N_j g_j v_{j,r} \right) \log \pi_\theta(a_{j,r} \mid \phi_{j,r}, \mathcal{A}_{j,r}). \end{aligned} \tag{9}$$

Each accepted rollout has $N_j > 0$. Cost weights sum to N_j, whereas utility weights sum to one when credit is assigned. The inner average normalizes the cost channel, and the N_j factor preserves the trajectory's total utility penalty. The outer average weights trajectories equally.

Following projected primal–dual optimization [29], we update the multiplier using the signed residual $q_j = U^{\text{ref}} - \epsilon - U_j$.

$$\mu_{\nu+1} = \Pi_{[0, \mu_{\max}]} \left(\mu_\nu + \eta_\mu K^{-1} \sum_{j=1}^{K} q_j \right). \tag{10}$$

An average utility shortfall increases μ_ν, strengthening subsequent utility penalties. A group that exceeds its utility target can decrease μ_ν. The signed residual drives this adjustment, while $g_j = [q_j]_+$ penalizes violating trajectories in the policy loss.

Updates use only complete current-policy groups with valid utilities, $N_j > 0$, and nonzero reward variance. For each accepted group, the training cycle proceeds as follows.

(1) Retrieve the task's fixed full-cache reference and collect K trajectories with the current policy head.
(2) Obtain terminal utilities and progress curves, then compute A_j, $w_{j,r}$, g_j, and $v_{j,r}$ from the recorded trajectories.
(3) Take one optimizer step on Equation 9 and update the dual variable using Equation 10.
(4) Collect the next group with the updated head.

Appendix A.4 gives the normalization and update eligibility rules.

5 Experiment

Our main experiments evaluate growing tool-interaction histories in software workflows, measuring completion, inference cost, and memory. Shared-context retrieval further tests whether compressed evidence remains available to later queries.

5.1 Benchmarks and Data Splits

(1) Terminal-Bench 2.1 [21] evaluates command-line tasks through tool interaction and task-specific verifiers. We use its fixed 89-task evaluation set. *(2) SWE-bench Verified* [10] evaluates repository-level issue resolution by testing the agent's patch. We use the 200-task subset adopted by CompactionRL [16]. *(3) SCBench* [18] evaluates retrieval utility preservation on four subsets, KV, Prefix-Suffix, RepoQA, and RepoQA+KV. Natural-language task instructions make up only a small fraction of the input. The vast majority consists of compression-sensitive JSON records, strings, and code, leaving little redundant prose. This tests whether compression preserves evidence for later queries.

5.2 Backbones and Runtimes

We select agent backbones by three criteria. For *(1) architecture*, we use full-attention Transformers with persistent KV caches to study compression as context and KV state grow. For *(2) capability*, we require demonstrated software-agent performance in public evaluations. For *(3) scale*, we target roughly 30B-class or smaller backbones within our compute budget. Consequently, we select Qwen3-Coder-30B-A3B-Instruct [27] and Devstral-Small-2-24B [22].

We run the frozen agent backbones with SGLang [42], using batch size one per replica and one replica per GPU. Agents use mini-swe-agent [35] with terminal tools. Harbor [6] runs Terminal-Bench tasks, and the official SWE-bench harness scores repository patches. Weights are FP8 on Terminal-Bench and AWQ INT4 with group size 32 on SWE-bench, with BF16 activations and KV throughout. Each agent backbone has its own policy head and frozen feature-normalization statistics. Sampling parameters, software versions, and time limits are specified in Appendix B.1. Segmentation thresholds are specified in Appendix A.1.

The SCBench retrieval adaptation uses Qwen3-30B-A3B-Instruct-2507 [26] and GLM-4-9B-Chat-1M [39]. They run multi-turn LLM dialogues with generated history in native Transformers, without agents or tools.

5.3 Baselines and Ablations

We choose baselines to cover text compression, KV compression, their serial composition, and agent-trajectory compression, alongside an uncompressed reference. *(1) Full-cache* retains the complete history and KV cache. *(2) LLMLingua-2* [24] uses its official API's default 50% target retention on observations or shared retrieval context, preserving instructions and tool-call structure. *(3) SnapKV* [17] retains at most 4,096 positions per layer and KV head, a capacity evaluated in its original study. *(4) Serial* first applies LLMLingua-2, computes and appends KV entries for the compressed new observation, and then applies SnapKV. *(5) AgentDiet* [34] compresses history through its reflection-and-rewrite procedure. Compressor settings and reflection parameters are specified in Appendix B.2.

On Qwen Terminal-Bench and SWE-bench, we test three ablation groups. *(1) Action ablations* restrict the policy to {`keep`, `drop`} or {`keep`, `evict`}. *(2) Feature ablations* mask text-derived or KV-derived inputs at creation and decision time while preserving the architecture and all three actions. *(3) Uniform credit* distributes cost credit equally across non-protected decisions. It distributes utility penalties equally across decisions eligible for subsequent generation, including `keep`. Each trajectory's assigned credit mass remains unchanged. Each of the five variants trains its own policy head. Appendix B.2 specifies the masks and credit support.

5.4 Training and Evaluation Protocol

For agents, we screen and sample a 200-task training pool from SWE-smith [36]. We train the compact policy head while keeping the backbone and feature normalization frozen. Training, validation, and the two test rosters have disjoint task identifiers. The head is shared across units within a backbone and trained on online rollout groups drawn from this pool, following Section 4.5. Each group contains one fixed full-cache reference and seven current-policy rollouts. Appendices A.3 and B.1 specify initialization, optimizer, progress-judge, and credit settings.

We select each backbone's checkpoint on held-out validation tasks, favoring higher official task success and lower mean backbone cost. The selected head is fixed before evaluating either test benchmark and remains fixed across runs. Main agent comparisons use three evaluation runs. Each method receives one evaluation trajectory per task per run. Retrieval splits are repository-disjoint.

5.5 Evaluation Metrics and Accounting

Task performance. We report *Success* as the number or percentage of officially successful Terminal-Bench tasks, resolved SWE-bench instances, or correct retrieval queries. Timeouts and errors count as failures, with all tasks or queries retained in the denominator.

Utility preservation. We pair each method with Full-cache by task ID for agents and by query for retrieval. Let n_{SS} count tasks or queries on which both succeed, and n_{SF} count Full-cache successes that the method fails to retain. Utility preservation (*UP*) is

$$\mathrm{UP} = \frac{n_{\mathrm{SS}}}{n_{\mathrm{SS}} + n_{\mathrm{SF}}}. \tag{11}$$

UP measures the fraction of Full-cache successes retained and is reported as a percentage. Additional successes on tasks or queries that Full-cache does not solve are outside this conditional denominator.

Table 1: Task-level quality and backbone cost in GPU-s/task. Success, UP, and Total show mean ± SD over 3 runs. *Prefill* includes append and recompute. Throughout, bold marks unrounded compressed-method optima, including ties.

	Terminal-Bench 2.1 (89 tasks)							SWE-bench Verified (200-task subset)						
Method	Success (%) ↑	UP (%) ↑	Prefill ↓	Decode ↓	Total ↓	Output (tokens/ task)	GPU-s/ 1k tokens ↓	Success (%) ↑	UP (%) ↑	Prefill ↓	Decode ↓	Total ↓	Output (tokens/ task)	GPU-s/ 1k tokens ↓
Qwen3-Coder-30B-A3B-Instruct														
Full-cache	11.24±1.12	100.00±0.00	0.58	31.79	32.37±0.63	3797.08	8.52	28.50±1.00	100.00±0.00	1.09	80.79	81.88±2.24	8511.56	9.62
LLMLingua-2	3.37±2.25	10.00±0.51	0.41	40.90	41.31±0.61	3892.99	10.61	6.00±0.50	17.54±1.15	0.80	69.26	70.06±0.56	7978.02	8.78
SnapKV	7.87±1.12	40.00±1.59	0.55	34.35	34.90±1.18	3306.90	10.56	24.50±0.50	52.63±1.37	0.85	64.56	65.40±1.27	8735.23	**7.49**
Serial	5.62±1.12	20.00±2.34	**0.37**	37.94	38.31±0.61	3860.85	9.92	2.50±0.50	8.77±1.94	**0.61**	**50.70**	**51.31**±0.94	6673.83	7.69
AgentDiet	11.24±1.12	50.00±3.76	0.80	32.68	33.48±0.60	3340.47	10.02	28.00±1.00	57.89±0.39	1.28	51.83	53.11±1.14	6153.38	8.63
LACING	**12.36**±2.25	**70.00**±3.87	0.54	**28.72**	**29.26**±0.71	3795.31	**7.71**	**28.50**±0.50	**61.40**±1.52	1.06	73.72	74.78±2.39	8886.93	8.41
Devstral-Small-2-24B														
Full-cache	21.35±1.12	100.00±0.00	7.25	577.53	584.78±7.31	9733.69	60.08	48.50±0.50	100.00±0.00	10.67	988.55	999.23±33.40	16475.84	60.65
LLMLingua-2	8.99±1.12	36.84±3.45	10.08	694.24	704.32±32.86	12251.12	57.49	7.50±0.50	13.40±1.57	9.93	536.96	546.90±6.40	9293.13	58.85
SnapKV	17.98±1.12	**73.68**±4.23	**5.75**	**362.87**	**368.62**±10.24	6893.06	53.48	37.50±1.00	59.79±1.86	10.09	771.88	781.97±3.30	15531.79	**50.35**
Serial	5.62±2.25	21.05±0.46	7.66	410.42	418.08±5.49	8180.24	**51.11**	6.00±0.50	12.37±0.40	10.62	631.97	642.60±16.32	12512.67	51.36
AgentDiet	17.98±1.12	**73.68**±5.07	12.55	585.20	597.74±19.16	10371.12	57.64	19.50±0.50	32.99±1.51	**8.18**	**240.94**	**249.12**±6.99	4411.29	56.47
LACING	**20.22**±1.12	**73.68**±4.17	8.59	532.49	541.09±13.83	9685.69	55.86	**47.50**±0.50	**74.23**±1.50	10.46	849.37	859.82±24.66	15650.21	54.94

Table 2: Memory–runtime trade-offs. Peak KV, TTFT, and TPOT show mean ± SD over 3 runs.

	Terminal-Bench 2.1 (89 tasks)						SWE-bench Verified (200-task subset)					
Method	Text (%) ↓	KV (%) ↓	Peak KV (GiB) ↓	TTFT (s) ↓	TPOT (ms/token) ↓	Overhead (s/task) ↓	Text (%) ↓	KV (%) ↓	Peak KV (GiB) ↓	TTFT (s) ↓	TPOT (ms/token) ↓	Overhead (s/task) ↓
Qwen3-Coder-30B-A3B-Instruct												
Full-cache	100.00	100.00	1.00±0.00	0.20±0.010	152.73±9.55	0.00	100.00	100.00	1.86±0.05	0.18±0.010	150.70±4.22	0.00
LLMLingua-2	**81.27**	81.27	0.78±0.01	0.20±0.010	148.53±8.85	**1.00**	**77.07**	77.07	1.52±0.05	**0.18**±0.003	**142.20**±4.49	**2.59**
SnapKV	100.00	70.20	0.54±0.01	0.26±0.004	186.12±7.59	1.20	100.00	**35.96**	0.53±0.02	0.20±0.010	143.76±6.87	3.70
Serial	82.32	**66.46**	**0.47**±0.00	0.18±0.003	125.77±4.94	2.08	77.13	40.33	**0.48**±0.00	0.22±0.020	183.92±7.32	6.79
AgentDiet	83.01	83.01	0.76±0.04	0.42±0.030	147.35±2.88	38.00	84.45	84.45	1.29±0.02	0.38±0.040	159.08±3.45	46.58
LACING	98.38	79.32	0.74±0.01	**0.17**±0.003	**113.50**±2.40	6.33	94.91	62.52	1.13±0.02	0.18±0.003	151.92±1.58	33.04
Devstral-Small-2-24B												
Full-cache	100.00	100.00	3.66±0.06	0.22±0.004	61.01±1.67	0.00	100.00	100.00	5.77±0.02	0.22±0.010	68.72±2.24	0.00
LLMLingua-2	74.70	74.70	3.64±0.05	0.21±0.003	56.03±2.43	6.45	68.17	68.17	4.73±0.11	0.21±0.020	63.74±2.80	**7.13**
SnapKV	100.00	50.34	1.10±0.04	0.26±0.010	102.24±2.74	**3.81**	100.00	19.56	1.06±0.02	**0.17**±0.003	**23.67**±1.02	12.53
Serial	**73.29**	**37.32**	**0.87**±0.02	**0.16**±0.010	105.34±3.78	12.76	69.90	**18.32**	**0.98**±0.04	0.19±0.010	26.13±0.79	30.71
AgentDiet	79.36	79.36	2.76±0.07	0.46±0.030	**52.33**±2.23	48.48	80.40	80.40	2.53±0.04	0.96±0.050	104.59±4.82	65.06
LACING	77.33	70.68	2.16±0.06	0.21±0.010	60.56±1.48	29.13	**67.46**	57.43	2.59±0.06	0.19±0.010	67.36±1.26	49.54

Resource metrics. *(1) Backbone GPU cost* uses a stage-wise roofline model [31] under fixed model and hardware profiles. By accounting for forward computation, weight reads, and KV traffic, it reflects the backbone resource effects of compression without tool-execution or orchestration delays. All agent methods within a backbone share an INT4 weight-reference profile independent of execution precision. We aggregate initial prefill, incremental append, decode, and recompute under the accounting protocol in Appendix B.3, which also specifies the effective compute and bandwidth calibration. Agent values are mean accounted GPU-seconds per task, excluding method overhead. Retrieval uses GPU-s/query and native calibration. *(2) Output* counts generated backbone tokens. *(3) GPU-s/1k* is $1000 \sum C / \sum N_{\text{out}}$, where N_{out} counts reported output tokens.

(4) Text retention and *(5) KV retention* measure retained text tokens and active KV bytes relative to each method's own uncompressed history. *(6) Peak KV* is the mean per-task peak GPU-resident active KV payload. Snapshot selection and task-level aggregation are specified in Appendix B.3.

(7) TTFT (time to first token) and *(8) TPOT* (time per output token) [43] measure backbone forward wall time. *(9) Overhead* reports separately timed method work, including compression and external reflection. These wall-time measurements are distinct from backbone GPU-seconds. Appendix B.3 specifies timer boundaries and averaging rules.

5.6 Trajectory Analysis

We analyze saved trajectories through *(1) action shares* over trajectory progress, *(2) task-paired changes* in agent steps, executed tool calls, and *Output*, *(3) cumulative backbone GPU cost along agent steps*, and *(4) recorded failure symptoms* in unsuccessful runs. Appendix B.4 gives the eligibility rules, normalization and evidence-coverage requirements used in Figures 4–7. Cost curves sum prefill, append, decode, and recompute under the Table 1 accounting, with completed tasks retaining their terminal cost in the fixed-roster mean. Figure 6 compares all six agent methods on their own observed trajectories. Final task success is reported in Table 1.

6 Results

6.1 Quality–Cost Trade-offs and Task Preservation

Success stays near Full-cache across settings. Table 1 ranks LACING highest among compressors in *Success* and *UP* across all four agent settings. It records 181 successes versus Full-cache's 183, with 7.5–14.0% lower backbone cost. Figure 3 shows it is the only tested compressor within the five-percentage-point success tolerance in every setting while lowering cost. Success stays within 1.2 points of Full-cache, with non-dominated cost and peak-KV trade-offs.

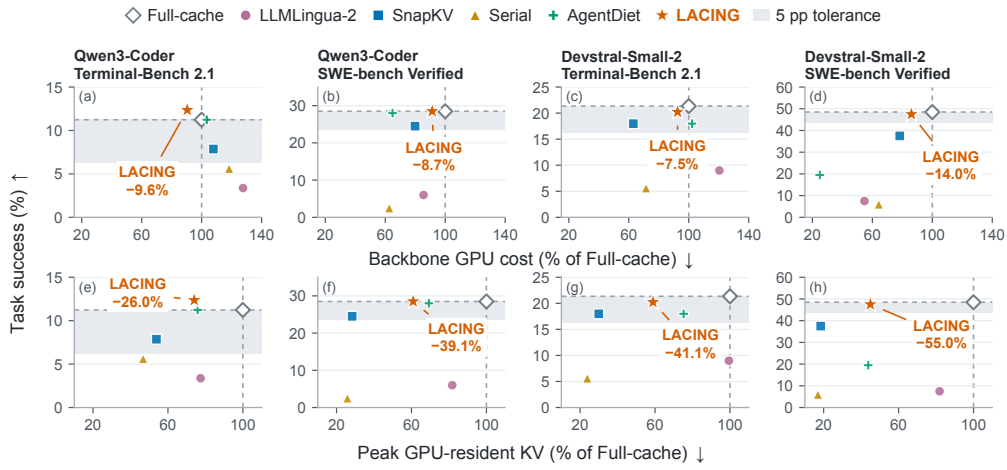


Figure 3: Quality–efficiency trade-offs, with resource axes relative to Full-cache at 100%. Shading marks a 5-percentage-point success tolerance below Full-cache.

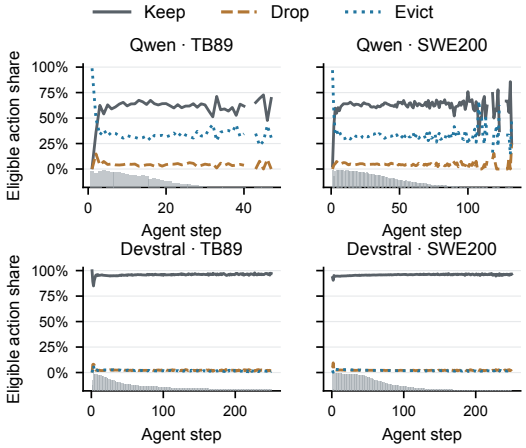


Figure 4: LACING action shares over eligible units, averaged equally across contributing tasks. Gaps in the Qwen curves occur at steps where all units are protected. Gray strips count contributing tasks.

SnapKV preserves more success than LLMLingua-2 in all four settings, favoring KV-side reduction over direct token deletion under these configurations. Its Devstral SWE-bench success nevertheless falls 11 points below Full-cache, versus one for LACING. Other methods offer distinct operating points. On Qwen SWE-bench, Serial costs 51.31 GPU-s/task but resolves five tasks, while AgentDiet resolves 56 at 53.11 GPU-s/task. LACING combines cost savings with consistently high task retention across both backbones and benchmarks.

Savings concentrate in decode. *Decode* accounts for 98.2–98.8% of LACING's cost. LLMLingua-2 compresses observations before encoding, whereas LACING uses normal backbone representations and subsequent attention evidence to select state for later interactions.

Table 3: Qwen action, feature, and credit ablations.

Method	Success ↑ (n/N, %)	UP ↑ (%)	Prefill ↓	Decode ↓	Total ↓	Output (tokens/ task)	GPU-s/ 1k tokens ↓
Terminal-Bench 2.1 (89 tasks)							
LACING	11/89 (12.4%)	**70.00**	**0.539**	**28.719**	**29.257**	3795.3	7.709
KV-only	**12/89 (13.5%)**	**70.00**	0.659	34.011	34.669	4286.4	8.088
Text-only	9/89 (10.1%)	60.00	0.759	36.064	36.823	4739.2	7.770
w/o text features	9/89 (10.1%)	50.00	0.721	35.173	35.895	4637.2	7.741
w/o KV features	8/89 (9.0%)	**70.00**	0.618	35.624	36.242	4741.0	**7.644**
Uniform credit	11/89 (12.4%)	50.00	0.784	38.940	39.724	5010.7	7.928
SWE-bench Verified (200-task subset)							
LACING	**57/200 (28.5%)**	**61.40**	1.060	73.719	74.779	8886.9	8.414
KV-only	55/200 (27.5%)	59.65	1.075	75.545	76.620	8793.7	8.713
Text-only	48/200 (24.0%)	45.61	1.042	78.484	79.525	8678.9	9.163
w/o text features	52/200 (26.0%)	56.14	**0.995**	**72.354**	**73.350**	8751.5	**8.381**
w/o KV features	50/200 (25.0%)	54.39	1.052	73.978	75.030	8932.4	8.400
Uniform credit	47/200 (23.5%)	54.39	1.105	76.771	77.876	9178.7	8.484

Its main benefit is reducing the state repeatedly accessed during generation, even where *Prefill* increases. On Devstral Terminal-Bench, *Prefill* rises from 7.25 to 8.59 GPU-s/task, while *Decode* falls from 577.53 to 532.49. The larger reduction in decode cost more than offsets the extra encoding cost.

Lower per-token cost offsets longer output. On Qwen SWE-bench, LACING matches 57 resolutions despite 4.4% more output. Cost per 1,000 tokens falls from 9.62 to 8.41 GPU-s, a 12.5% reduction that outweighs the longer output. Total cost falls by 8.7%.

LACING leads compressors in utility preservation. In the paired Qwen SWE-bench example, LACING and Full-cache both resolve 57 tasks, with 35 shared successes. Relative to Full-cache, the policy gains 22 successes and loses 22 reference successes, giving 61.4% *UP*, versus AgentDiet's 33 retained successes with 57.9% *UP*. On Devstral SWE-bench, it retains 72 of 97 reference successes with 74.2% *UP*, versus SnapKV's 58 with 59.8% *UP*. Reporting both metrics distinguishes maintained completion rates from retention of particular solutions.

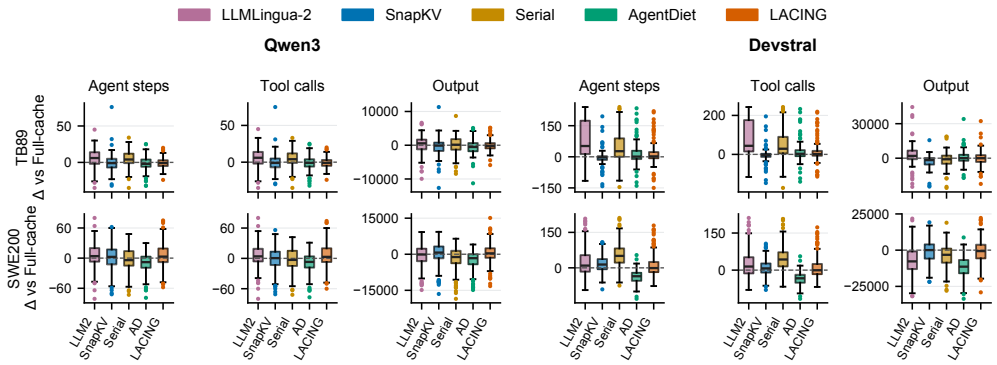


Figure 5: Task-paired differences computed as method minus Full-cache in steps, tool calls, and the *Output* metric from Table 1. Boxes span the interquartile range (IQR), with median lines, 1.5-IQR whiskers, and all outliers shown.

Table 4: Retrieval preservation on SCBench. Cells give *Success* counts, *UP* percentages, and *Total* in GPU-s/query, in that order.

	Qwen3-30B-A3B-Instruct-2507					GLM-4-9B-Chat-1M				
Method	KV ($N = 500$)	Prefix–Suffix ($N = 500$)	RepoQA ($N = 160$)	Mix-KV ($N = 128$)	Mix-RepoQA ($N = 128$)	KV ($N = 500$)	Prefix–Suffix ($N = 500$)	RepoQA ($N = 160$)	Mix-KV ($N = 128$)	Mix-RepoQA ($N = 128$)
Full-cache	500/100/55.998	10/100/53.540	118/100/41.876	117/100/15.010	100/100/33.227	85/100/27.114	222/100/38.179	108/100/43.021	106/100/12.691	84/100/34.341
LLMLingua-2	2/0.4/26.783	1/0/32.824	17/13.6/**36.638**	0/0/14.363	16/16/**30.069**	0/0/13.337	10/1.8/12.013	10/6.5/20.228	0/0/7.992	9/8.3/17.052
SnapKV	11/2.2/38.019	1/0/39.767	17/14.4/44.021	10/8.5/15.115	8/8/31.927	4/4.7/14.173	7/2.3/13.828	9/7.4/11.801	9/8.5/4.749	3/3.6/9.104
Serial	0/0/**23.784**	0/0/**32.621**	7/5.9/37.314	0/0/**12.625**	2/2/36.605	0/0/**6.531**	1/0.5/**6.195**	4/1.9/**9.720**	0/0/**3.613**	1/1.2/**7.286**
LACING	**492**/**98.4**/55.694	**11**/**20**/53.216	**117**/**97.5**/41.170	**114**/**96.6**/14.574	**98**/**98**/32.647	**79**/**92.9**/26.944	**210**/**85.1**/37.855	**101**/**93.5**/45.100	**99**/**92.5**/12.564	**84**/**95.2**/33.777

6.2 Cross-View Allocation and Compression Decisions

Compression patterns differ by backbone. Table 2 shows backbone-specific text–KV allocations. Qwen retains 94.91–98.38% of text but 62.52–79.32% of KV bytes, preserving readable history while reducing cached state. Devstral reduces both views more extensively, retaining 67.46–77.33% of text and 57.43–70.68% of KV. Across settings, *Peak KV* falls 26.0–55.0%.

Figure 4 connects these allocations to different intervention patterns. Qwen begins with an eviction burst, then favors `keep`, followed by `evict` and `drop`. Equal averaging over supported steps gives `keep` shares of 60.2% and 61.7%, versus Devstral's 96.0% and 95.9%. Each backbone uses one fixed head across both benchmarks.

Sparse interventions can yield cumulative compression. Under independent decisions with constant `keep` probability p, a unit facing n eligible opportunities has probability $1 - p^n$ of at least one compression. High per-decision retention can thus coexist with cumulative compression. Devstral's eligible decisions extend to step 250, versus Qwen's 47 and 132. Its keep-dominated SWE-bench policy retains 67.46% of text and 57.43% of KV bytes. Action shares count unit-level opportunities, whereas retention counts surviving tokens and bytes along the resulting history. The support strips track contributing tasks. Smaller late-step support makes the averages more sensitive to individual trajectories.

TTFT falls in three settings. On Qwen Terminal-Bench, *TTFT* falls by 13.0%, and *TPOT* falls from 152.73 to 113.50 ms/token, a 25.7% reduction. Other *TPOT* values stay near Full-cache. Separately timed *Overhead* is 6.33–49.54 seconds/task, 23.9–83.3% below AgentDiet.

6.3 Action, Feature, and Credit Ablations

Joint actions improve the quality–cost balance. Table 3 reports 57 SWE-bench resolutions at 74.78 GPU-s/task for the full policy, versus KV-only's 55 at 76.62 and Text-only's 48 at 79.53. Text-only also lowers *UP* from 61.40% to 45.61%. Joint selection therefore improves success, reference-task retention, and cost over either single-side action space in this setting. On Terminal-Bench, joint selection matches KV-only's 70% *UP* at 29.26 versus 34.67 GPU-s/task, with 11 versus 12 successes. KV-only outperforms Text-only on both benchmarks, supporting text preservation with selective KV reduction.

Both feature views support task success. Feature ablations preserve all three actions, separating the available choices from the information used to select them. Removing text or KV features reduces SWE-bench resolutions from 57 to 52 or 50 and Terminal-Bench successes from 11 to 9 or 8. The complete representation costs 1.43 GPU-s/task more than the no-text-feature policy on SWE-bench and resolves five additional tasks. Together, the two feature views improve selection within the same action space.

Localized credit improves preservation and efficiency. Uniform credit replaces the combined localized cost and utility allocation with equal-weight allocation. On Terminal-Bench, localized and uniform credit both solve 11 tasks. Localization raises *UP* from 50% to 70% and lowers cost by 26.3%, from 39.72 to 29.26 GPU-s/task. Uniform credit produces 32.0% more *Output*, with 2.8% higher *GPU-s/1k*. Its larger total cost reflects both more generated work and higher cost per output token. On SWE-bench, localization raises resolutions from 47 to 57 while lowering backbone cost from 77.88 to

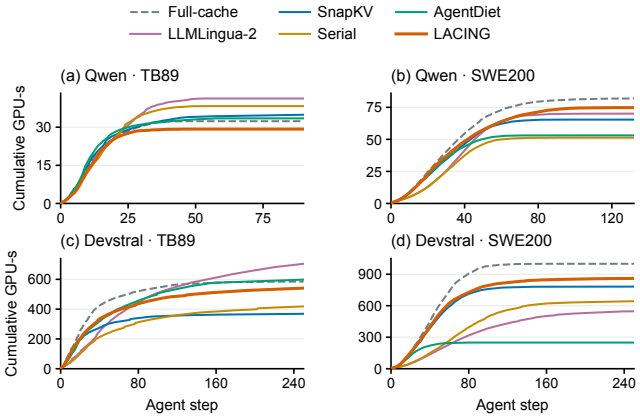


Figure 6: Mean cumulative backbone cost across steps.

74.78 GPU-s/task. Differentiating decisions by downstream savings and progress improves preservation and resource use.

6.4 Trajectory Effects and Execution Symptoms

Long tails dominate Devstral's mean step increases. Figure 5 pairs each method with Full-cache by task. On Qwen SWE-bench, 112/200 LACING trajectories use more steps, with median increases of three steps, three calls, and 378.5 output tokens. Lower cost thus accompanies a broad shift toward more interaction. Qwen Terminal-Bench instead has median step/call decreases of one.

Devstral's means are more strongly influenced by long trajectories. On Terminal-Bench, the mean step increase is 17.61 versus a median of two. The ten largest increases add 1,610 steps, while the other 79 tasks use 43 fewer. On Devstral SWE-bench, mean steps rise 9.28 despite a median decrease of one and 825.6 fewer mean output tokens. Savings thus coexist with both broadly increased interaction and long tails that dominate aggregate step counts.

Savings persist beyond the initial phase. Figure 6 shows LACING below Full-cache in all four panels after the initial phase. On both Terminal-Bench backbones, LLMLingua-2 is initially below Full-cache but finishes above it. Later interactions overtake its early advantage, motivating a full-trajectory view of compression. On Devstral, LACING accumulates work after the reference plateaus and still finishes lower. These curves connect the quality–cost operating points to the timing of resource use along each method's observed workload. Backbone cost depends on generated tokens and active KV state, so more interaction steps do not necessarily mean higher cumulative inference cost.

LACING records fewer timeout-associated SWE-bench failures than other compressors. Figure 7 records 3.5% and 4.5% of the Qwen and Devstral rosters, versus 36.5–89.0% and 9.0–72.0% for alternatives. Full-cache records 2.0% and 0%. LACING stays closest to the reference timeout profile while reducing memory and backbone cost.

Exact repeated replies distinguish another aspect of execution. Serial has the highest Devstral repetition rates, 28.1% and 59.5%, versus LACING's 21.3% and 13.5%. The latter is near Full-cache's 12.5%, while AgentDiet reaches 2.5%. SWE-bench protocol parsing

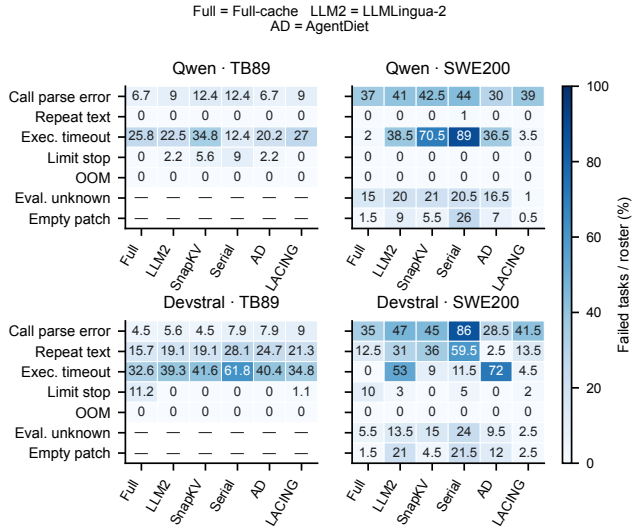


Figure 7: Unsuccessful tasks with each symptom as a percentage of the full roster. Categories inapplicable to Terminal-Bench scenarios are shown as dashes.

errors remain more frequent than with Full-cache. Symptoms can overlap within an unsuccessful trajectory.

6.5 Retrieval Preservation in Shared Contexts

Retrieval utility is best preserved among compressors. Table 4 presents retrieval results on the four SCBench subsets. LACING leads the compressors in *Success* and *UP* across all ten model–task groups. It retains over 92% of Full-cache successes on key-value and code tasks, versus at most 16% for baselines.

Their compression-sensitive content includes JSON and other key–value records, string prefixes and suffixes, and code. With little redundant prose, deletion can remove evidence needed by later queries. This preservation-first operating point lowers cost by 0.7% on average across the ten groups.

7 Conclusion

In this paper, we formulated prompt and KV-cache compression as a joint sequential decision problem for multi-turn agents. LACING links text and cached states through stable memory anchors and learns a unified policy over preservation, text deletion, and head-wise KV reduction. Separate cost and utility credit signals connect compression decisions to resource savings and task progress over later interactions. Experiments with two backbones on Terminal-Bench 2.1 and SWE-bench Verified show reduced backbone GPU cost and peak KV memory at near-Full-cache aggregate task success. On Qwen, localized credit improves utility preservation and lowers cost relative to uniform credit. On SCBench, the policy retains over 92% of Full-cache successes on compression-sensitive key-value and code tasks. Overall, our work offers a promising direction for efficient multi-turn agents by managing textual history and cached states jointly, with compression guided by cumulative cost and future task utility.

References

[1] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting Language Models to Compress Contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 3829–3846. https://aclanthology.org/2023.emnlp-main.232/

[2] Wenjie Du, Li Jiang, Keda Tao, Xue Liu, and Huan Wang. 2026. Which Heads Matter for Reasoning? RL-Guided KV Cache Compression. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. PMLR, 26480–26500. https://proceedings.mlr.press/v306/du26e.html

[3] Shaoting Feng, Yuhan Liu, Hanchen Li, Xiaokun Chen, Samuel Shen, Kuntai Du, Zhuohan Gu, Rui Zhang, Yuyang Huang, Yihua Cheng, Jiayi Yao, Qizheng Zhang, Ganesh Ananthanarayanan, and Junchen Jiang. 2025. EvicPress: Joint KV-Cache Compression and Eviction for Efficient LLM Serving. *arXiv preprint arXiv:2512.14946v1* (2025). https://arxiv.org/abs/2512.14946v1

[4] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2024. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. In *International Conference on Learning Representations*. https://openreview.net/forum?id=uNrFpDPMyo

[5] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. In *International Conference on Learning Representations*. https://openreview.net/forum?id=uREj4ZuGJE

[6] Harbor Framework Team. 2026. Harbor: A Framework for Evaluating and Optimizing Agents and Models in Container Environments. Software, version 0.22.0. doi:10.5281/zenodo.20953922

[7] Weizhong Huang, Jinchao Zhang, and Xiawu Zheng. 2026. CommitKV: Lifecycle-Aware KV Cache Compression via Commit Transitions for Multi-Turn Agents. *arXiv preprint arXiv:2608.07855* (2026). https://arxiv.org/abs/2608.07855

[8] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 13358–13376. https://aclanthology.org/2023.emnlp-main.825/

[9] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 1658–1677. https://aclanthology.org/2024.acl-long.91/

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations*. https://openreview.net/forum?id=VTF8yNQM66

[11] Dongwon Jo, Jiwon Song, Yulhwa Kim, and Jae-Joon Kim. 2026. FastKV: Decoupling of Context Reduction and KV Cache Compression for Prefill-Decoding Acceleration. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 32167–32186. https://aclanthology.org/2026.findings-acl.1610/

[12] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. 2026. ACON: Optimizing Context Compression for Long-horizon LLM Agents. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. PMLR, 55321–55375. https://proceedings.mlr.press/v306/kang26b.html

[13] Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. 2025. KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., 167563–167591. doi:10.52202/085713-5585

[14] Minsoo Kim, Arnav Kundu, Han-Byul Kim, Richa Dixit, and Minsik Cho. 2026. EpiCache: Episodic KV Cache Management for Long-Term Conversation on Resource-Constrained Environments. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. PMLR, 58057–58078. https://proceedings.mlr.press/v306/kim26ag.html

[15] Ping Li, Trevor J. Hastie, and Kenneth W. Church. 2006. Very Sparse Random Projections. In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 287–296. doi:10.1145/1150402.1150436

[16] Yujiang Li, Zhenyu Hou, Yi Jing, Jie Tang, and Yuxiao Dong. 2026. CompactionRL: Reinforcement Learning with Context Compaction for Long-Horizon Agents. *arXiv preprint arXiv:2607.05378* (2026). https://arxiv.org/abs/2607.05378v2

[17] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 22947–22970. doi:10.52202/079017-0722

[18] Yucheng Li, Huiqiang Jiang, Qianhui Wu, Xufang Luo, Surin Ahn, Chengruidong Zhang, Amir H. Abdi, Dongsheng Li, Jianfeng Gao, Yuqing Yang, and Lili Qiu. 2025. SCBench: A KV Cache-Centric Analysis of Long-Context Methods. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=gkUyYcY1W9

[19] Taowen Tony Liu, Jeffrey T. H. Wong, Can Xiao, Bowen Yang, Hao Mark Chen, and Yiren Zhao. 2026. AgentKV: Phase-Aware KV Eviction for Agentic LLMs. *arXiv preprint arXiv:2609.14872v1* (2026). https://arxiv.org/abs/2609.14872v1

[20] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. Association for Computing Machinery, New York, NY, USA, 38–56. doi:10.1145/3651890.3672274

[21] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. *arXiv preprint arXiv:2601.11868* (2026). https://arxiv.org/abs/2601.11868

[22] Mistral AI. 2025. Devstral Small 2 24B Instruct 2512. Model card. Retrieved 24 October 2026 from https://huggingface.co/mistralai/Devstral-Small-2-24B-Instruct-2512

[23] Luca Moschella, Laura Manduchi, and Ozan Sener. 2026. Learning to Evict from Key-Value Cache. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. PMLR, 90470–90496. https://proceedings.mlr.press/v306/moschella26a.html

[24] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 963–981. doi:10.18653/v1/2024.findings-acl.57

[25] Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. 2025. MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation. In *Proceedings of the ACM on Web Conference 2025*. Association for Computing Machinery, New York, NY, USA, 2366–2377. doi:10.1145/3696410.3714805

[26] Qwen Team. 2025. Qwen3-30B-A3B-Instruct-2507. Model card. Retrieved 25 October 2026 from https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct-2507

[27] Qwen Team. 2025. Qwen3-Coder-30B-A3B-Instruct. Model card. Retrieved 24 October 2026 from https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct

[28] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300* (2024). https://arxiv.org/abs/2402.03300

[29] Chen Tessler, Daniel J. Mankowitz, and Shie Mannor. 2019. Reward Constrained Policy Optimization. In *International Conference on Learning Representations*. https://openreview.net/forum?id=SkfrvsA9FX

[30] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 63897–63911. https://proceedings.mlr.press/v267/wang25bx.html

[31] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (April 2009), 65–76. doi:10.1145/1498765.1498785

[32] Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. 2025. ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization. *arXiv preprint arXiv:2509.13313* (2025). https://arxiv.org/abs/2509.13313v3

[33] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2025. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. In *International Conference on*

Learning Representations. https://arxiv.org/abs/2410.10819

[34] Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2026. Reducing Cost of LLM Agents with Trajectory Reduction. *arXiv preprint arXiv:2509.23586v2* (2026). https://arxiv.org/abs/2509.23586v2

[35] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 50528–50652. doi:10.52202/079017-1601

[36] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. SWE-smith: Scaling Data for Software Engineering Agents. *arXiv preprint arXiv:2504.21798* (2025). https://arxiv.org/abs/2504.21798

[37] Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik Chaudhari, George Karypis, and Huzefa Rangwala. 2025. AgentOccam: A Simple Yet Strong Baseline for LLM-Based Web Agents. In *International Conference on Learning Representations*. https://arxiv.org/abs/2410.13825v2

[38] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X

[39] Z.ai. 2024. GLM-4-9B-Chat-1M. Model card. Retrieved 25 October 2026 from https://huggingface.co/zai-org/glm-4-9b-chat-1m

[40] Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, and Jitao Sang. 2026. Memory as Action: Autonomous Context Curation for Long-Horizon Agentic Tasks. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 19149–19164. https://aclanthology.org/2026.findings-acl.956/

[41] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 34661–34710. doi:10.52202/075280-1506

[42] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37. 62557–62583. doi:10.52202/079017-2000

[43] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 193–210. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[44] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*. https://arxiv.org/abs/2307.13854v4

A Method Details

A.1 Segmentation and Unit Lifecycle

Newly appended tokens are segmented before compression. The segmenter distinguishes task and control content, model reasoning, tool actions, and environment feedback, then maps these fields to contiguous token intervals. Tool-call syntax is separated from command payloads. Commands remain atomic, even when they exceed the target length. Compressible reasoning and observations target 32-token units with an eight-token boundary-search tolerance. Segments shorter than eight tokens merge when structural boundaries permit. System instructions, task content, submission markers, harness controls, and protocol syntax are permanently protected. Open command–response pairs are also protected, following Section 4.2.

Aids are allocated monotonically. An aid is never derived from an array index, reused after deletion, or changed by text or KV compaction. Canonical intervals and token content remain fixed after creation. At decision point t, units created in turn t have age zero and are forced to `keep`; units from turn $t-1$ become eligible. A single feasibility mask combines content protection, open-pair protection, and the one-step protection of newly created units.

For active anchors, the runtime maintains the current text interval $J_{i,t}$ and head-indexed support $\mathbf{Z}_{i,t}$. Active text units are compacted into a gap-free buffer in canonical order. KV entries are physically packed but retain their canonical position labels. A `drop` removes the entire unit from both model views and records its aid as a tombstone. Its current text interval becomes $\perp$, and all of its surviving headwise supports become empty, including supports reduced by earlier evictions. Retired aids and removed head–token cells remain inactive. Canonical positions determine the coordinates of future appends. New suffix states append directly to the surviving compressed state without a full re-prefill.

A.2 Cross-View Feature Schema

Creation-time features. When a suffix is appended, we capture keys after final-layer RoPE, final-layer values, and hidden states after final normalization. Each tensor is pooled over the unit tokens by mean, maximum, first token, and last token. Key and value pooling retains each backbone's KV-head by head-dimension coordinates, and hidden-state pooling retains its residual coordinates. We also record the mean token norm for each key and value head and four position features: normalized start and end, length/256, and $\log(1+\text{length})/\log(257)$. The resulting primitive has width $D = 2(4H_{\text{KV}}d_h) + 4d_{\text{model}} + 2H_{\text{KV}} + 4$. With $d_h = 128$, Devstral has $H_{\text{KV}} = 8$, $d_{\text{model}} = 5{,}120$, and $D = 28{,}692$. Qwen3-Coder has $H_{\text{KV}} = 4$, $d_{\text{model}} = 2{,}048$, and $D = 12{,}300$. The primitive is pooled in FP32 and standardized coordinate-wise with frozen statistics (μ_x, σ_x), flooring standard deviations at 10^{-6}. Values are clipped to $[-10, 10]$ before a fixed sparse random projection [15] maps them to $\phi_i^{\text{create}} \in \mathbb{R}^{512}$. For the backbone's input width D, each projection column samples $s = \text{round}(\sqrt{D})$ coordinates without replacement, with equally likely signs, magnitude $\sqrt{D/(512s)}$, and seed 42.

Decision-time prompt features. The five prompt coordinates have the following fixed order.

(1) Directional containment in later observed units. For units of at least 24 tokens, let G_i be the set of contiguous token trigrams. The feature is the running maximum of $|G_i \cap G_k|/|G_i|$ over later units k observed in the current history. Both units must meet the 24-token threshold. Shorter units and units without an eligible later match receive zero.
(2) The command parser's write-event indicator.
(3) An indicator that at least one write-target path was extracted.
(4) The logarithmic path count $\log(1 + n_{\text{paths}})$.
(5) The logarithmic later-write count $\log(1 + n_{\text{later}})$.

Write paths are extracted from shell redirection, `tee`, `cp`/`mv` destinations, and recognized in-place edits. Relative paths are normalized against the task's fixed workspace, and duplicate target paths are counted once. For each target path, we count subsequent observed writes to that path. A command with multiple targets uses the minimum of these counts for n_{later}, so the aggregate is positive only after every target has a later write. Unreliable extraction or an empty target set gives $n_{\text{later}} = 0$. Non-command units have

Table 5: Nine KV-demand features in input order.

No.	Coordinate	Meaning
1	m_{all}	All-history mean attention.
2	M_{all}	All-history mean of head maxima.
3	m_{recent}	Recent mean attention.
4	M_{recent}	Recent mean of head maxima.
5	$\mathbf{1}[n_{\text{all}} > 0]$	All-history availability.
6	$\mathbf{1}[n_{\text{recent}} > 0]$	Recent-window availability.
7	$\omega/2$	Window length divided by two.
8	$\log(1 + n_{\text{all}})$	Log all-history query count.
9	$\log(1 + n_{\text{recent}})$	Log recent-window query count.

zero-valued command-state coordinates. These updates use only observed history.

Decision-time KV features. Nine features summarize subsequent use of the surviving KV cells. We retain final-layer query rows whose canonical positions follow the unit and exclude attention internal to it. Query heads are grouped by their associated GQA KV head. For each valid query row and query head, we sum attention probabilities over the unit's surviving positions in the associated KV head and divide by its support size. We then take the mean and maximum over query heads for that row, followed by an average over valid rows in the chosen horizon. Let $m_{\text{all}}, M_{\text{all}}$ denote these two statistics over all valid rows since the current headwise membership was established. The corresponding recent statistics $m_{\text{recent}}, M_{\text{recent}}$ use at most the two most recent completed decision steps.

Let n_{all} and n_{recent} count valid query rows in these horizons, and let $\omega \in \{0, 1, 2\}$ be the number of completed steps represented in the recent window. Table 5 lists the nine coordinates in input order. Unavailable mean and maximum statistics are encoded as zero, with the corresponding availability indicator set to zero. An observed zero attention value instead has availability one. Changing $\mathbf{Z}_{i,t}$ resets both horizons and their counts.

Lifecycle features. The remaining three coordinates, in order, are $\log(1 + \text{age})$, the common per-head KV survival ratio $|Z^h_{i,t}|/|I_i|$, and $\log(1 + n_{\text{evict}})$. Age counts completed decision steps, and n_{evict} counts prior evictions of the same aid. Together, the five prompt, nine KV, and three lifecycle coordinates form $\phi^{\text{state}}_{i,t} \in \mathbb{R}^{17}$. The six unbounded counts use $\log(1 + x)$, and the recent-window length is divided by two. Ratios, attention statistics, and Boolean indicators retain their original scale. No additional fitted normalization is applied to these 17 coordinates.

A.3 Policy Masks and Headwise Execution

The shared MLP is $529 \to 256 \to 256 \to 3$, with LayerNorm and SiLU after each hidden layer. Its output weights and biases are zero-initialized, giving equal probability to feasible actions. The same head is shared across units and decisions. Protected units admit only `keep`. `evict` additionally requires a common per-head support size of at least eight, attention evidence from queries after the unit, and a feasible assignment that preserves each token in at least one KV head.

All unit features and masks are constructed from the same pre-compression state. The runtime samples one action per active aid, validates the complete batch, and only then mutates state. For an evicted unit with current common support size n, every KV head retains exactly $\lfloor n/2 \rfloor$ positions. A capacity-constrained assignment first covers all positions in I_i, favoring cells with higher recent attention. Remaining capacity is filled by each head's own ranking from its associated GQA query-head group in the final layer. Eviction is masked when the reduced capacities cannot simultaneously satisfy exact per-head halving and $\bigcup_h Z^h_{i,t} = I_i$.

Execution gathers each head using its selected canonical positions and applies that headwise layout at every transformer layer. All heads retain equal packed lengths even when their logical positions differ. The same atomic commit compacts surviving text, retires dropped aids, increments eviction counts, and resets attention statistics for changed memberships.

A.4 Decision-Level Credit

Cost attribution. In Equations 6–7, the event set $\mathcal{E}_{j,r}$ starts strictly after decision $r = (i, t)$ and ends with the last forward before the next non-keep action on aid_i, or the terminal forward. Agent-step indices locate decisions. Forward-event indices locate the individual prefill, append, decode, and recompute operations between them.

The counterfactual holds generated tokens, other actions, and method overhead fixed. Under headwise eviction it restores removed packed KV bytes rather than changing token identities. Under `drop` it restores the unit's text length and the KV support present just before deletion, retaining the effects of earlier evictions. A `keep` decision has zero marginal because its counterfactual changes no resources. If all marginals vanish, every trainable decision has $w_{j,r} = 1$.

Utility attribution. We align progress curves on the regular four-step grid through the longer trajectory's termination. Once a trajectory has ended, later grid points use its terminal score. For $d_j(t) = p_j(t) - p^{\text{ref}}(t)$, a regression requires two consecutive grid points below -0.1. The first of these points defines t^*_j. If no regular-grid candidate remains and both trajectories terminate in the same scoring interval, a terminal difference below -0.1 also defines a regression, with t^*_j equal to the later terminal step.

We exclude only decisions confirmed to have no subsequent model-generation opportunity. The set $\mathcal{W}_j$ contains the remaining trainable `drop` and `evict` decisions with $\max(0, t^*_j - W) \le t_r < t^*_j$. Equation 8 assigns normalized recency weights within this window. Window and decay parameters are given in Appendix B.1. Without persistent regression, credit is uniform over eligible compression actions. For a detected regression with a positive violation and an empty window, the virtual-terminal fallback requires t^*_j beyond the last completed generation. It distributes credit uniformly over earlier compression actions with confirmed subsequent generation opportunities. If no such actions exist, or an empty window does not meet the fallback condition, all utility weights are zero. `Keep`

receives no utility penalty. Trajectories invalidated by protocol or infrastructure failure are excluded.

Group normalization and update eligibility. For the K current-policy rollouts, subtracting the group mean and dividing by the population standard deviation gives

$$A_j = \frac{R_j - \bar{R}}{\sigma_R}, \quad \bar{R} = K^{-1} \sum_j R_j, \\ \sigma_R^2 = K^{-1} \sum_j (R_j - \bar{R})^2. \tag{12}$$

The fixed full-cache reference supplies C^{ref} but is excluded from these group statistics. An accepted group must be complete, and every rollout must have valid utility feedback, $N_j > 0$ trainable decisions, and actions sampled by the current policy. Incomplete and exact zero-variance groups do not update the head. Each accepted group produces one optimizer step and is consumed once. The surrogate uses recorded actions directly, without importance-ratio clipping. Rollouts sampled by earlier heads do not train the updated policy.

B Experimental Details

B.1 Models, Runtimes, and Training

Table 6 lists the frozen evaluation settings. Policy training uses eight independent tensor-parallel-size-one collectors on NVIDIA A100 80GB GPUs and a single FP32 learner. Agent evaluation uses SGLang, batch size one per replica, and one replica per GPU. Activations and the KV cache remain in BF16. Models and tokenizers are fixed within each setting.

The training pool contains 200 screened and sampled SWE-smith tasks. Each backbone has eight held-out validation tasks disjoint from training. Checkpoint selection follows Section 5. The selected head remains fixed across the three main evaluation runs, with one trajectory per method and task per run.

Terminal-Bench 2.1 uses mini-swe-agent 2.4.6 and Harbor 0.22.0 on the fixed 89-task roster. The retained-text working-context limit is 65,536 tokens, while the canonical uncompressed history may grow to 262,144 positions. Each model response is capped at 10,240 tokens and each trajectory at 250 model calls. Commands have a 60-second limit and model requests a 1,800-second limit. The agent and verifier retain each task's official timeout with multiplier one. SWE-bench uses a 3,600-second agent limit and a 300-second evaluation limit.

Optimization and feedback. Each online training group contains one fixed full-cache reference and seven rollouts sampled from the current policy. We update the policy head in FP32 using AdamW with learning rate 2×10^{-4}, $\beta = (0.9, 0.95)$, $\epsilon_{\text{Adam}} = 10^{-8}$, and zero weight decay. Group eligibility follows Appendix A.4. Training utility uses $\lambda_{\text{succ}} = 0.65$, giving weights 0.65 to official success and 0.35 to terminal progress. The utility tolerance is $\epsilon = 0.05$. Localized utility credit uses a five-step window, $W = 5$, with recency decay $\gamma = 0.8$. The dual starts at 0.5, with learning rate 0.5 and projection to $[0, 2]$, updated once per accepted group.

Progress is scored every four agent steps and at termination using three independently ordered evaluations from `gpt-5.6-luna`, without additional reasoning mode. The judge receives the task requirements and anonymized reference and candidate execution prefixes. Each prefix is scored independently on $[0, 1]$, where zero denotes no useful progress and one denotes task completion. The rubric rewards concrete task advancement and penalizes premature stopping, looping, and avoidable termination. The three scores for each prefix are averaged. Curve alignment and regression-window rules follow Appendix A.4.

B.2 Baseline and Ablation Configurations

Agent baselines share Table 6's settings.

LLMLingua-2 uses `microsoft/llmlingua-2-xlm-roberta-large-meetingbank`. Its target retention is 50% for each newly appended nonempty tool observation. Newlines and the characters `?`, `!`, and `.` are forced to remain; digits receive no separate protection. System content, task instructions, assistant reasoning, commands, roles, tool names, and tool-call identifiers are unchanged.

SnapKV uses a capacity of 4,096 positions per layer and KV head, including a 64-token observation window, with kernel size 5 and average pooling. It runs after incremental prefill and before decoding only when the cache exceeds the capacity. Serial applies LLMLingua-2 to the new tool observation, computes and appends KV entries for the compressed observation, and then applies the same SnapKV configuration. Neither SnapKV arm replays the history after compression.

AgentDiet uses its released reflection and rewrite procedure with a `gpt-5-mini` reflection endpoint configured for low reasoning, 128,000 input tokens, and 8,192 output tokens per reflection call. SCBench uses Full-cache, LLMLingua-2, SnapKV, and Serial with unchanged budgets, applying text compression to shared retrieval context.

Action and feature ablations. Text-only permits `keep` and `drop`; KV-only permits `keep` and `evict`. Both retain the three-logit head and mask the excluded action. Feature ablations retain all three actions and the 529-dimensional head input. Without text features, we zero the pooled hidden-state blocks of the creation primitive and the five decision-time prompt coordinates. Without KV features, we zero the pooled K/V blocks and per-head norm statistics, the nine KV-demand coordinates, the KV survival ratio, and the eviction count. Static masking follows frozen normalization and precedes the unchanged 512-dimensional projection. Dynamic coordinates are zeroed after encoding. Position, length, and age information remains available in both ablations. The normalizer, projection, and head capacity are unchanged.

Uniform credit. Cost weights are one for all non-protected decisions, preserving the full method's total cost weight of N_j for trajectory j. For utility, we first compute the full method's assigned penalty mass, then distribute that mass equally across all non-protected decisions with a subsequent-generation opportunity, including `keep`. Decisions confirmed to have no such opportunity are excluded; uncertain cases remain eligible. This allocation replaces the localized, recency-weighted support and preserves any zero assigned mass. The objective, optimizer, and dual-update schedule remain unchanged.

Table 6: Evaluation settings for the frozen backbones.

Setting	Configuration
Agent backbones	Qwen3-Coder-30B-A3B-Instruct and Devstral-Small-2-24B. Each has a frozen normalizer and policy head.
Terminal-Bench	FP8 weights, BF16 activations and KV, temperature 0.15.
SWE-bench	AWQ INT4/group-32 weights, BF16 activations and KV, temperature 1.
Agent sampling	Top-p = 1, base seed 42, one evaluation trajectory per task per run.
SCBench	Qwen3-30B-A3B-Instruct-2507 (AWQ INT4) and GLM-4-9B-Chat-1M (BF16). Native Transformers, greedy multi-turn dialogues with generated history.

Table 7: Batch-one calibration for agent backbones.

Backbone	Stage	Compute	Memory
Qwen	Prefill	0.542430	0.220189
	Append	0.419145	0.208280
	Decode	0.250000	0.230565
Devstral	Prefill	0.497088	0.150079
	Append	0.448643	0.135593
	Decode	0.250000	0.166359

B.3 Cost and Measurement

For a forward event e, the stage-wise roofline estimate is

$$\widehat{T}_e = \max\left(\frac{f_e}{P_e^{\text{eff}}}, \frac{B_e^{\text{weights}}}{W_e^{\text{weights}}} + \frac{B_e^{\text{KV}}}{W_e^{\text{KV}}}\right), \tag{13}$$

where f_e is estimated floating-point work, the two B_e terms are weight and active-cache read volumes, and the denominators are effective compute throughput and memory bandwidths. Reported cost aggregates the accounted initial prefill, incremental append, decode, and recompute terms of each trajectory. For presentation, initial prefill, append, and recompute are grouped as *Prefill*.

The roofline calibration uses NVIDIA A100 80GB PCIe specifications and stage-specific effective compute and memory efficiencies measured at batch size one. Table 7 lists the multipliers for the shared INT4 reference of each agent backbone. Effective compute throughput is the stage's compute multiplier times 312 TFLOP/s; the weight and KV read terms both use the memory multiplier times 1.935 TB/s. Calculations retain the full-precision calibration values. All agent methods within a backbone use the same INT4 weight reference and recorded active KV bytes. SCBench uses model-native profiles. Backbone GPU-seconds measure inference resource cost rather than end-to-end wall time. Totals exclude policy-head work, compression, tools, verification, and external services.

Retention and peak KV.. Retention uses pre-decode snapshots of completed requests. Each snapshot records retained text tokens and active KV bytes together with their uncompressed counterparts on the same method's history. Within a task, we divide the sum of retained quantities by the sum of the corresponding uncompressed quantities, then average the task ratios with equal weight. Peak KV is the maximum active GPU-resident K/V payload observed throughout a task, including pre-compression, post-compression, and decode states. The reported value averages these task peaks. CPU cache, temporary copies, model weights, activations, and unused reserved capacity are excluded.

Forward latency. For each complete request, TTFT sums the recorded backbone forward intervals needed to produce the first output token's logits, including prefill, append, and any required recompute. TPOT sums forward time for subsequent output tokens and divides by the number of generated tokens minus one. Single-token responses have no TPOT. We average complete-request measurements within each task, then average task means with equal weight. Independently timed method work is excluded from both latency metrics.

Method overhead. Disjoint timers cover primitive pooling, attention statistics, policy evaluation, action execution, and headwise KV repacking. For external baselines, the timers include LLMLingua-2 compression and AgentDiet reflection services as applicable. We sum these intervals within each task and report their mean wall time per task, separately from forward latency and backbone GPU-seconds.

B.4 Trajectory Analysis Protocol

Paired trajectory measures. We match each method to Full-cache by task ID and retain the fixed 89- or 200-task roster. Agent steps count observed model-response steps, while executed tool calls count tool-return events. Counted output follows the *Output* definition in Table 1. Figure 5 plots method-minus-Full-cache differences, with medians, interquartile boxes, 1.5-interquartile-range whiskers, and all remaining outliers.

Action aggregation. Discretionary decisions exclude protected units, forced keeps, and choices without another legal action. Action shares are computed within task-steps, then averaged equally over tasks with at least one such decision at that step. Support strips show this contributing-task count.

Cumulative backbone cost. At each observed agent step, we accumulate prefill, append, decode, and recompute GPU-seconds on the method's own trajectory. Costs follow Appendix B.3. We average over the fixed 89- or 200-task roster, retaining each task's terminal cumulative value after it ends. The endpoints equal the mean Total costs in Table 1; method overhead is excluded. Figure 6 compares all six methods on observed workloads, including their different generated tokens and tool observations. Final official success rates are reported alongside cost in Table 1.

Recorded symptoms. For each method, a symptom counts at most once per unsuccessful task. The denominator remains the full 89- or 200-task roster, and categories can overlap. Repeated text requires consecutive identical nonempty assistant replies. Tool-call parsing errors are rejections by the Qwen XML or Devstral argument-JSON

parser. Errors are logged natively or identified by replaying the same parsers on saved replies. A witnessed rejection establishes a positive task label; a negative label requires coverage of all completed responses. Missing tool calls alone and command execution failures do not count as protocol parsing errors. Execution timeouts, runtime-limit stops, out-of-memory errors, and unavailable evaluations remain separate categories. Empty patches apply only to SWE-bench. A percentage is displayed only when the category has complete evidence coverage. Unknown or inapplicable categories receive a dash; zero requires complete coverage and no observed occurrence among unsuccessful tasks.